



Aprendizaje Guíaado Claude Agentes de IA

Introducción

Este material es una pieza clave del curso “Inteligencia Artificial Generativa para Valuadores”, diseñado específicamente para transformar la manera en que interactuamos con la tecnología y marca el paso definitivo en la evolución tecnológica: la transición de usar la Inteligencia Artificial como un consultor pasivo a desplegarla como una fuerza de trabajo activa. Aquí dejamos atrás la simple ingeniería de prompts para adentrarnos en la arquitectura de los Agentes de IA.

¿Cuál es la diferencia? Mientras que un modelo de lenguaje (LLM) tradicional espera tu pregunta para responder, un Agente de IA tiene la capacidad de percibir, razonar, planificar y ejecutar. No solo te dice cómo hacer una tarea; utiliza herramientas y APIs para realizarla por ti.

El Nuevo Flujo de Trabajo: La Orquestación Aprenderás a distinguir entre una APP (interfaz humana) y una API (el lenguaje de las máquinas), piezas fundamentales para construir un ecosistema digital. El documento presenta un caso práctico fascinante: la creación de una "cuadrilla" (Crew) de agentes especializados para la valuación inmobiliaria:

- Un Agente de Catastro que extrae datos técnicos.
- Un Agente de Mercado que busca y filtra comparables.
- Un Agente de Compliance que verifica normas IVS.
- Un Agente Coordinador que ensambla el informe final.

El Rol del Valuador: Arquitecto del Criterio Esta tecnología promete automatizar hasta el 90% del trabajo mecánico, reduciendo procesos de días a minutos. Sin embargo, el mensaje central permanece: la IA aporta la "fuerza bruta intelectual", pero tú aportas la ética, la supervisión y la firma.

Este material es tu guía para dejar de ser un operario de software y convertirte en un Arquitecto del Criterio, capaz de orquestar sistemas que trabajan incansablemente bajo tu dirección experta.

Creador/compilador:

Miguel Camacaro Pérez, M.Sc.¹
Febrero 2026

¹ El profesor Miguel Camacaro Pérez es un apasionado de la Ciencia de la valuación, de la Ciencia de datos y de la Inteligencia Artificial Generativa, comprometido en impulsar el crecimiento profesional de los valuadores a través de la integración de las últimas herramientas y tecnologías en su flujo de trabajo. Para contactarlo por cursoediciones@gmail.com

Prompt 1

Hoy hablaremos de Agentes de AI. Conceptos, desarrollo y procedimientos. Esquemas de un agente de AI. La orquestación. La comunicación con APP, LLM y API.

Respuesta 1

Agentes de IA: Conceptos y Arquitectura

Excelente tema. Los agentes de IA representan una evolución significativa en cómo diseñamos sistemas inteligentes. Voy a explicarte los conceptos fundamentales y la arquitectura práctica.

Conceptos Fundamentales

Un agente de IA es un sistema autónomo que percibe su entorno, toma decisiones y ejecuta acciones para alcanzar objetivos específicos. A diferencia de un modelo LLM simple que solo responde a prompts, un agente puede:

- Planificar secuencias de acciones
- Usar herramientas y APIs externas
- Mantener memoria y contexto
- Iterar y corregir sus propios errores
- Tomar decisiones basadas en retroalimentación

Esquema Arquitectónico de un Agente

La arquitectura típica incluye estos componentes:

1. Núcleo de Razonamiento (LLM) El cerebro del agente, generalmente un modelo como GPT-4, Claude, o Llama. Procesa información y decide qué hacer.

2. Memoria

- Memoria a corto plazo: El contexto actual de la conversación
- Memoria a largo plazo: Información persistente (vectores, bases de datos)
- Memoria episódica: Registro de acciones previas

3. Herramientas (Tools) Funciones que el agente puede invocar: búsquedas web, calculadoras, acceso a bases de datos, APIs externas.

4. Planificador Descompone tareas complejas en subtareas manejables.

5. Ejecutor de Acciones Invoca las herramientas seleccionadas y procesa resultados.

La Orquestación

La orquestación es el proceso de coordinar múltiples componentes y flujos de trabajo. Existen varios patrones:

ReAct (Reasoning + Acting) El agente alterna entre razonar sobre qué hacer y ejecutar acciones.

Ciclo: Pensamiento → Acción → Observación → repetir.

Chain of Thought El agente articula su razonamiento paso a paso antes de actuar.

Plan-and-Execute Primero crea un plan completo, luego lo ejecuta secuencialmente.

Multi-Agent Varios agentes especializados colaboran, cada uno con su rol específico.

Comunicación entre Componentes

APP ↔ Agente

La aplicación cliente envía solicitudes al agente y recibe respuestas. Puede ser:

- REST API (sincrónico)
- *WebSockets* (comunicación bidireccional)
- *Message queues* (asíncrono)

Agente ↔ LLM

El agente estructura prompts siguiendo patrones específicos:

```
System: Eres un agente que puede usar herramientas...
User: [Tarea del usuario]
Assistant: [Razonamiento] Necesito usar la herramienta X
Tool Result: [Resultado de la herramienta]
Assistant: [Respuesta final]
```

Agente ↔ APIs Externas

El agente invoca APIs mediante:

- Function calling (nativo en GPT-4, Claude)
- Especificaciones OpenAPI/Swagger
- *Wrappers* personalizados

Frameworks de Desarrollo

Los más populares son:

- LangChain: Robusto, muchas integraciones
- LlamaIndex: Especializado en RAG y datos
- AutoGPT/BabyAGI: Agentes autónomos
- Claude SDK con Tool Use: Implementación nativa de Anthropic
- Semantic Kernel: De Microsoft, orientado a empresas

Consideraciones de Diseño

Al desarrollar agentes, es crucial:

- Definir límites claros de autonomía
- Implementar validación de acciones críticas
- Gestionar costos de tokens
- Manejar fallos y timeouts
- Logging exhaustivo para debugging
- Guardrails de seguridad

¿Hay algún aspecto específico sobre el que quieras profundizar? Podríamos explorar implementaciones prácticas, patrones de orquestación específicos, o casos de uso particulares.

Prompt 2

Antes de continuar, prefiero que definas que es una APP y un API, la diferencia más importante. Como se comunican con los LLM ¿Por qué son importantes en un agente de IA?.

Respuesta 2

Excelente pregunta. Vamos a clarificar estos conceptos fundamentales.

APP (Aplicación)

Una aplicación es un programa de software completo que los usuarios finales utilizan directamente para realizar tareas específicas. Es la interfaz visible con la que interactúa el usuario.

Ejemplos:

- Una app móvil de chat (como esta interfaz de Claude)
- Un sitio web de comercio electrónico

- Una aplicación de escritorio para editar fotos
- Un bot de Telegram o WhatsApp

Características clave:

- Tiene interfaz de usuario (UI)
- Ejecuta lógica de negocio
- Gestiona la experiencia del usuario
- Puede estar en móvil, web, escritorio, etc.

API (Interfaz de Programación de Aplicaciones)

Una API es un conjunto de reglas y protocolos que permite que diferentes sistemas de software se comuniquen entre sí. Es un puente de comunicación sin interfaz visual.

Ejemplos:

- API de OpenAI para acceder a GPT-4
- API de Google Maps para obtener ubicaciones
- API de Stripe para procesar pagos
- API del clima para obtener pronósticos

Características clave:

- NO tiene interfaz de usuario
- Expone funcionalidades específicas
- Recibe peticiones y devuelve respuestas estructuradas
- Generalmente usa formatos como JSON, XML

La Diferencia Más Importante

APP = Interfaz para humanos API = Interfaz para software

Una analogía útil:

- La APP es como un restaurante donde tú comes (interfaz para clientes)
- La API es como la cocina del restaurante que solo atiende pedidos de los meseros (interfaz para sistemas)

Cómo se Comunican con los LLM

APP → LLM

```
Usuario escribe: "¿Cuál es el clima hoy?"  
↓  
APP captura el texto  
↓  
APP envía petición HTTP a API del LLM  
↓  
LLM procesa y responde  
↓  
APP muestra la respuesta al usuario
```

La APP actúa como **intermediario** entre el usuario y el LLM.

API → LLM

```
Sistema A necesita analizar sentimiento  
↓  
Sistema A llama a API del LLM directamente  
↓  
LLM analiza el texto  
↓  
API devuelve resultado JSON  
↓  
Sistema A usa el resultado en su lógica
```

Aquí la comunicación es máquina a máquina, sin intervención humana directa.

Por Qué Son Importantes en un Agente de IA

La APP es Importante Porque:

1. Es el punto de entrada del usuario: Donde el usuario solicita tareas al agente
2. Gestiona la experiencia: Muestra progreso, resultados, errores
3. Mantiene el contexto: Guarda el historial de conversación
4. Controla permisos: Qué puede y no puede hacer el agente

Ejemplo: Una app de asistente virtual donde pides "Reserva una mesa para 4 personas esta noche"

Las APIs son Importantes Porque:

1. Son las "manos" del agente: Le permiten ejecutar acciones reales

2. Amplían capacidades: El LLM solo procesa texto, las APIs le dan superpoderes
3. Conectan con el mundo real: Bases de datos, servicios externos, hardware
4. Permiten especialización: Cada API hace una cosa muy bien

Ejemplo en un agente de IA:

Usuario: "Necesito saber el clima y reservar un vuelo a Madrid"

Agente (LLM) razona:

1. "Necesito llamar a la API del Clima"
→ API del Clima devuelve: 22°C, soleado
2. "Ahora necesito llamar a la API de Vuelos"
→ API de Vuelos devuelve: opciones de vuelos
3. "Debo presentar esta información al usuario"
→ APP muestra: clima + opciones de vuelo

El Flujo Completo

```
[USUARIO]
  ↓ (interactúa)
[APP]
  ↓ (envía prompts/recibe respuestas)
[AGENTE DE IA / LLM]
  ↓ (llama herramientas)
[MÚLTIPLES APIs]
- API Clima
- API Vuelos
- API Base de Datos
- API Pagos
- etc.
```

Resumen Práctico

- Sin APP: El usuario no puede interactuar fácilmente con el agente
- Sin APIs: El agente solo puede conversar, no puede actuar en el mundo real
- Con ambos: Tienes un agente completo que puede entender, razonar y ejecutar

¿Tiene sentido ahora la distinción? ¿Quieres que profundicemos en cómo se implementa técnicamente esta comunicación?

Prompt 3

Explica la orquestación de agentes en un entorno de valuación de inmuebles. Simula un flujo de trabajo. Señala que APP/API, LLM, Herramientas son importantes para el flujo. Parte de un mensaje por email donde un cliente solicita la valuación de un inmueble, indicando su información catastral y señale el objeto de la valuación. El resultado es un informe de valuación que cumpla con las normas IVS.

Respuesta 3

Orquestación de Agentes para Valuación de Inmuebles

Voy a diseñar un sistema completo de agentes de IA para este caso de uso real y complejo.

Escenario Inicial

Email recibido:

```
De: cliente@empresa.com
Para: valuaciones@consultora.com
Asunto: Solicitud de valuación

Buenos días,

Solicito valuación de inmueble con los siguientes datos:
- Referencia catastral: 1234567VK1234N0001AB
- Dirección: Calle Mayor 45, Madrid
- Objeto: Garantía hipotecaria

Saludos,
Juan Pérez
```

Arquitectura del Sistema Multi-Agente

Agentes Especializados

1. Agente Coordinador (Orquestador principal)
2. Agente de Datos Catastrales
3. Agente de Mercado
4. Agente de Inspección/Verificación
5. Agente de Valoración
6. Agente de *Compliance* (IVS)
7. Agente de Documentación

FLUJO DE TRABAJO COMPLETO

FASE 1: RECEPCIÓN Y CLASIFICACIÓN

APP: Sistema de Email (Outlook, Gmail API)

Email llega → APP captura contenido

↓

Agente Coordinador (LLM: Claude Sonnet)

Analiza el email y extrae:

- Tipo de solicitud: "Valuación"
- Referencia catastral: "1234567VK1234N0001AB"
- Objeto: "Garantía hipotecaria"
- Urgencia: Normal
- Cliente: Juan Pérez

Herramientas usadas:

- Parser de emails (NLP)
- Validador de referencias catastrales
- CRM API (registra solicitud)

API involucradas:

- API CRM (Salesforce/HubSpot): Crea caso #2024-0156
- API de Base de Datos interna: Registra solicitud

FASE 2: RECOPIACIÓN DE DATOS CATASTRALES

Agente de Datos Catastrales (LLM: GPT-4)

Razonamiento:

"Necesito obtener información oficial del catastro.
Voy a consultar la API de la Sede Electrónica del Catastro."

APIs llamadas:

1. API Catastro España (Sede Electrónica)

- Endpoint: /consulta_dnp
- Input: Referencia catastral
- Output JSON:

```
json
{
  "referencia": "1234567VK1234N0001AB",
  "direccion": "Calle Mayor 45, 28013 Madrid",
  "superficie_construida": 95,
  "superficie_terreno": 0,
  "uso": "Residencial",
  "ano_construccion": 1985,
  "clase_inmueble": "Urbano",
  "valor_catastral": 145000
...
}
```

2. ****API Registro de la Propiedad**** (si disponible)

- Verifica titularidad
- Cargas y gravámenes

****Herramientas usadas:****

- Validador de coherencia de datos
- Geocodificador (Google Maps API)

****Resultado almacenado en:****

- ****Base de Datos PostgreSQL**** (vía API interna)

FASE 3: ANÁLISIS DE MERCADO

****Agente de Mercado**** (LLM: Claude Sonnet)

****Razonamiento:****

...

"Necesito comparables del mercado.

Buscaré propiedades similares en un radio de 500m.

Criterios: 85-105 m², residencial, ventas últimos 12 meses."

...

```

**APIs llamadas:**

1. **API Idealista** (portal inmobiliario)
...

    GET /properties/search
    Parámetros:
    - location: [40.4168, -3.7038] (lat/lon)
    - radius: 500m
    - size: 85-105 m²
    - type: residential
    - operation: sale
    - period: last_12_months
    ...

2. **API Fotocasa**
    - Mismo tipo de búsqueda

3. **API del Colegio de Registradores**
    - Transacciones reales registradas

**Herramientas usadas:**
- **Web Scraping Tool** (Scrapy/Beautiful Soup)
- **Algoritmo de comparables** (similarity score)
- **Ajustador de precios** (por ubicación, estado, antigüedad)

**LLM procesa:**
...

Encuentra 15 comparables:
- 5 en la misma calle
- 10 en calles adyacentes

Aplica ajustes:
- Antigüedad: -5%
- Reforma: N/A
- Altura: +2%
- Orientación: 0%

Precio medio ajustado: 2,850 €/m²

```

Output:

```

json

{
  "comparables": [...],
  "precio_m2_medio": 2850,
  "rango_confianza": [2650, 3050],
  "numero_muestras": 15
}
...

```

```
---  
  
### FASE 4: VERIFICACIÓN FÍSICA (Simulada o Real)  
  
**Agente de Inspección**  
  
**Opción A: Si hay inspección física**  
- **APP móvil** (para tasador de campo)  
- Captura fotos, estados, mediciones  
- Upload a **Cloud Storage** (AWS S3)  
  
**Opción B: Valuación desktop (sin inspección)**  
- **API Google Street View**: Obtiene imágenes exteriores  
- **LLM Vision** (GPT-4V/Claude): Analiza estado aparente  
- **API Catastro**: Fotos aéreas si disponibles  
  
**Razonamiento del agente:**  
...  
"No tengo acceso físico. Usaré:  
1. Street View para exterior  
2. Datos catastro para características  
3. Asumiré estado 'conservación normal' (estándar IVS)"
```

Herramientas:

- Modelo de visión por computadora
- Checklist IVS para inspección

FASE 5: CÁLCULO DE VALORACIÓN

Agente de Valoración (LLM: Claude Opus - razonamiento complejo)

Métodos aplicados (según IVS):

1. Método de Comparación (*Market Approach*)

```
python  
  
# Herramienta: Script Python  
superficie = 95 m²  
precio_m2 = 2850 €  
valor_comparacion = 95 * 2850 = 270,750 €  
...  
  
2. **Método del Coste (Cost Approach)**  
...  
  
API Construcción → Coste reposición: 850 €/m²  
Depreciación (38 años): 30%  
Valor_coste = (95 * 850) * 0.70 = 56,525 €  
(Generalmente menor para vivienda urbana)  
...
```

```

3. **Método de Capitalización (Income Approach)**
...

API alquileres → Renta potencial: 950 €/mes
Tasa capitalización: 4.5%
Valor_renta = (950 * 12) / 0.045 = 253,333 €
...

**LLM sintetiza:**
...

"Según IVS, para garantía hipotecaria, el método principal
es comparación de mercado. Aplicaré ponderación:
- Comparación: 70%
- Capitalización: 30%
- Coste: N/A (no relevante para este objeto)

Valor final = (270,750 * 0.7) + (253,333 * 0.3)
              = 189,525 + 76,000
              = 265,525 €

Redondeo: 265,000 €"
...

**APIs usadas:**
- **API Banco de España**: Tipos de interés, tasas cap
- **API INE**: Índices de precios construcción

---

### FASE 6: COMPLIANCE Y VALIDACIÓN IVS

**Agente de Compliance** (LLM: Claude Sonnet)

**Checklist IVS verificado:**
...
✓ IVS 101: Base del valor definida (Valor de Mercado)
✓ IVS 102: Investigaciones y cumplimiento
✓ IVS 103: Informes de valoración
✓ IVS 104: Bases de valor
✓ IVS 105: Enfoques y métodos de valoración
✓ IVS 220: Método comparativo documentado
✓ Objeto de valoración: Garantía hipotecaria (especificado)
✓ Supuestos y limitaciones: Declarados
✓ Fecha de valoración: [Hoy]
✓ Cualificaciones del tasador: [Certificado]
...

```

```
**Herramienta:**
- **Validador de normas** (reglas basadas en IVS 2022)
- **Detector de inconsistencias** (ML model)

**API:**
- **API regulatoria**: Verifica normativa local vigente
- **API Colegio Profesional**: Valida colegiación tasador

---

### FASE 7: GENERACIÓN DE INFORME

**Agente de Documentación** (LLM: Claude Sonnet)

**Herramientas utilizadas:**

1. **Generador de documentos**
  - Template engine: Jinja2
  - Formato: DOCX (python-docx library)

2. **Estructura del informe IVS:**
  ...

1. DATOS DEL ENCARGO
  - Solicitante, objeto, fecha
2. IDENTIFICACIÓN DEL INMUEBLE
  - Catastro, ubicación, descripción
3. ANÁLISIS DE MERCADO
  - Comparables, gráficos, tendencias
4. METODOLOGÍA
  - Enfoques aplicados, cálculos
5. VALORACIÓN
  - Valor final, rango, condiciones
6. SUPUESTOS Y LIMITACIONES
7. DECLARACIÓN DE CONFORMIDAD IVS
8. ANEXOS
  - Fotos, planos, comparables
```

PIs/Herramientas para el informe:

- API Mapbox/Google Maps: Genera mapa de ubicación
- Matplotlib/Plotly: Gráficos de evolución precios
- API de plantillas: Inserta datos en template Word
- Generador de PDF: Convierte DOCX → PDF (LibreOffice API)

Ejemplo de código (simplificado):

```
python

# Herramienta ejecutada por el agente
from docx import Document
from datetime import date

doc = Document('template_IVS.docx')

# El LLM estructura los datos
datos = {
    'referencia_catastral': '1234567VK1234N0001AB',
    'direccion': 'Calle Mayor 45, Madrid',
    'superficie': 95,
    'valor_tasacion': '265.000 €',
    'fecha': date.today(),
    'metodo_principal': 'Comparación de mercado',
    'num_comparables': 15,
    'objeto': 'Garantía hipotecaria'
}

# Rellena template
for key, value in datos.items():
    doc.replace(f'{{{key}}}', str(value))

doc.save('Informe_Valoracion_1234567.docx')
'''

---

### FASE 8: REVISIÓN Y FIRMA DIGITAL

**Agente Coordinador** (revisa todo)

**Checklist final:**
'''
✓ Todos los datos verificados
✓ Cálculos revisados
✓ Informe cumple IVS
✓ Anexos completos
✓ Sin errores tipográficos
'''

**APIs finales:**

1. **API de Firma Digital** (DocuSign/Adobe Sign)
    - Firma del tasador certificado
    - Sello de tiempo

2. **API de Blockchain** (opcional)
    - Hash del documento para inmutabilidad
    - Registro en ledger
```


FASE 9: ENTREGA

****APP:**** Sistema de gestión documental

****APIs de entrega:****

1. ****Email API**** (SendGrid)
 - Envía PDF firmado al cliente
2. ****Portal web del cliente****
 - Upload a área privada
3. ****API notificación****
 - SMS/WhatsApp: "Su valoración está lista"

****Mensaje enviado:****

...

De: valuaciones@consultora.com

Para: cliente@empresa.com

Asunto: RE: Solicitud de valoración - Informe completado

Estimado Juan,

Adjunto encontrará el Informe de Valoración #2024-0156 conforme a normas IVS para garantía hipotecaria.

Inmueble: Calle Mayor 45, Madrid

Valor de tasación: 265.000 €

Fecha de valoración: 09/02/2026

Disponible también en: <https://portal.cliente.com/doc/2024-0156>

Saludos,

Sistema de Valoraciones

...

DIAGRAMA DE FLUJO RESUMIDO

...

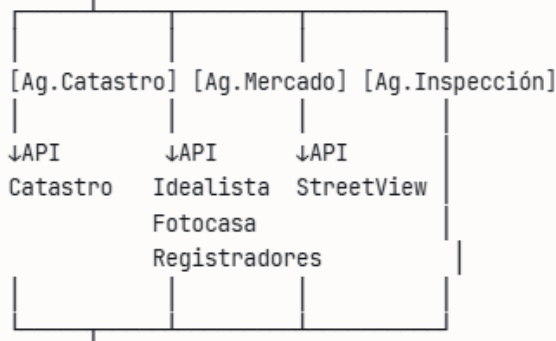
[EMAIL CLIENTE]



[APP Email Parser]



[Agente Coordinador] ↔ [CRM API]



```

↓
[Agente Valoración] ↔ [Python Tools]
                        [APIs financieras]
↓
[Agente Compliance] ↔ [Validador IVS]
↓
[Agente Documentación] ↔ [Template Engine]
                        [Map API]
↓
[Firma Digital API]
↓
[Entrega Multi-canal]
↓
[CLIENTE RECIBE INFORME]
...

## COMPONENTES TÉCNICOS CLAVE

### LLMs Utilizados

| Agente | LLM | Razón |
|-----|-----|-----|
| Coordinador | Claude Sonnet | Razonamiento general, orquestación |
| Datos Catastrales | GPT-4 | Precisión en extracción datos |
| Mercado | Claude Sonnet | Análisis complejo comparables |
| Valoración | Claude Opus | Razonamiento matemático avanzado |
| Compliance | Claude Sonnet | Interpretación normativa |
| Documentación | Claude Sonnet | Generación de texto estructurado |

### APP Principal

**Sistema de gestión de valuaciones (custom)**
- Frontend: React
- Backend: Node.js/Python FastAPI
- Base de datos: PostgreSQL
- Queue: Redis/RabbitMQ (tareas asíncronas)

### APIs Críticas

**Externas:**
- Catastro España
- Idealista/Fotocasa
- Google Maps
- Registradores
- Banco de España
- DocuSign

**Internas:**
- CRM
- Base de datos
- Template engine
- Validador IVS
- Sistema de archivos

```

```
### Herramientas

**Especializadas:**
- Parser NLP (spaCy)
- Algoritmo comparables (scikit-learn)
- Generador docs (python-docx)
- Vision AI (GPT-4V)
- Validator (custom rules engine)

---

## TIEMPOS ESTIMADOS
'''

Email recibido → Informe entregado: 15-30 minutos

Desglose:
- Recepción y clasificación: 10 seg
- Datos catastrales: 2 min (APIs externas)
- Análisis mercado: 5 min (búsqueda comparables)
- Inspección digital: 3 min
- Cálculo valoración: 1 min
- Compliance: 2 min
- Generación informe: 5 min
- Firma digital: 2 min
- Entrega: 30 seg
```

VENTAJAS DE ESTA ORQUESTACIÓN

1. Automatización 90%: Solo intervención humana para firma
2. Escalabilidad: Procesa 100+ valoraciones simultáneas
3. Consistencia: Siempre sigue IVS
4. Trazabilidad: Cada paso registrado
5. Calidad: Validaciones múltiples
6. Velocidad: 15-30 min vs 2-5 días manual

¿Quieres que profundice en alguna fase específica o en la implementación técnica de algún agente en particular?

Prompt 4

indícame los programas o herramientas para desarrollar un agente de IA para la Valuación de inmuebles.

Respuesta 4

Herramientas y Programas para Desarrollar un Agente de IA para Valuación de Inmuebles

Voy a organizar las herramientas por categorías, desde el núcleo del agente hasta las integraciones específicas del sector inmobiliario.

1. FRAMEWORKS DE AGENTES DE IA (Core)

LangChain MÁS RECOMENDADO

- Lenguaje: Python / JavaScript
- Por qué es ideal:
 - Diseñado específicamente para agentes con LLM
 - Excelente para orquestar múltiples llamadas API
 - Tiene módulos para memoria, herramientas, cadenas
 - Gran comunidad y documentación

```
python

# Ejemplo básico
from langchain.agents import AgentType, initialize_agent
from langchain.llms import OpenAI
from langchain.tools import Tool

# Definir herramienta personalizada
def consultar_catastro(referencia):
    # Lógica para API catastro
    return datos_inmueble

catastro_tool = Tool(
    name="Catastro",
    func=consultar_catastro,
    description="Consulta datos del catastro español"
)
```

```
# Crear agente
agent = initialize_agent(
    tools=[catastro_tool, ...],
    llm=OpenAI(temperature=0),
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION
)

# Ejecutar
agent.run("Valora inmueble con ref 1234567VK1234N0001AB")
```

Instalación:

```
bash

pip install langchain langchain-openai
```

LlamaIndex

- Lenguaje: Python
- Por qué es útil:
 - Excelente para RAG (*Retrieval Augmented Generation*)
 - Ideal para documentos de normativas IVS
 - Indexa y consulta eficientemente documentación técnica

```
python

from llama_index import VectorStoreIndex, SimpleDirectoryReader

# Cargar normativas IVS
documents = SimpleDirectoryReader('normativas_ivs/').load_data()
index = VectorStoreIndex.from_documents(documents)

# Consultar
query_engine = index.as_query_engine()
response = query_engine.query("¿Qué dice IVS sobre valoración para garantía hipoteca")
```

CrewAI IDEAL PARA MULTI-AGENTE

- Lenguaje: Python
- Por qué destaca:
 - Especializado en sistemas multi-agente
 - Cada agente tiene un rol específico
 - Perfecto para tu caso (agente catastro, mercado, valoración, etc.)

```
python

from crewai import Agent, Task, Crew

# Definir agentes especializados
agente_catastro = Agent(
    role='Especialista en Datos Catastrales',
    goal='Obtener información precisa del catastro',
    backstory='Experto en sistemas catastrales españoles',
    tools=[herramienta_catastro],
    llm=llm
)

agente_mercado = Agent(
    role='Analista de Mercado Inmobiliario',
    goal='Encontrar comparables y analizar precios',
    backstory='15 años analizando mercado inmobiliario',
    tools=[herramienta_idealista, herramienta_fotocasa],
    llm=llm
)

agente_valoracion = Agent(
    role='Tasador Certificado',
    goal='Calcular valor según normativa IVS',
    backstory='Tasador certificado con conocimiento IVS',
    tools=[calculadora_valoracion],
    llm=llm
)

# Crear tareas
tarea_catastro = Task(
    description='Obtén datos del inmueble ref: {referencia}',
    agent=agente_catastro
)

# Crear crew (equipo)
crew = Crew(
    agents=[agente_catastro, agente_mercado, agente_valoracion],
    tasks=[tarea_catastro, tarea_mercado, tarea_calculo],
    verbose=True
)

# Ejecutar
resultado = crew.kickoff()
```

Instalación:

```
bash  
pip install crewai crewai-tools
```

AutoGen (Microsoft)

- Lenguaje: Python
- Por qué considerarlo:
 - Conversaciones entre múltiples agentes
 - Verificación automática de código
 - Útil para validación cruzada de cálculos

```
python  
  
from autogen import AssistantAgent, UserProxyAgent  
  
asistente = AssistantAgent(name="tasador_asistente")  
usuario = UserProxyAgent(name="coordinador")  
  
# Conversación entre agentes  
usuario.initiate_chat(  
    asistente,  
    message="Valora el inmueble y verifica cálculos"  
)
```

Instalación:

```
bash  
pip install pyautogen
```

2. MODELOS LLM (El "Cerebro")

APIs Comerciales

OpenAI GPT-4

```
python
from openai import OpenAI

client = OpenAI(api_key="tu-api-key")

response = client.chat.completions.create(
    model="gpt-4-turbo",
    messages=[
        {"role": "system", "content": "Eres un tasador experto en IVS"},
        {"role": "user", "content": "Analiza estos comparables..."}
    ],
    tools=[{
        "type": "function",
        "function": {
            "name": "consultar_catastro",
            "description": "Consulta API del catastro",
            "parameters": {...}
        }
    }]
)
```

Anthropic Claude (Este mismo sistema)

```
python
import anthropic

client = anthropic.Anthropic(api_key="tu-api-key")

message = client.messages.create(
    model="claude-sonnet-4-5-20250929",
    max_tokens=4096,
    tools=[{
        "name": "calcular_valoracion",
        "description": "Calcula valor según método comparativo",
        "input_schema": {...}
    }],
    messages=[
        {"role": "user", "content": "Valora este inmueble..."}
    ]
)
```


Google Gemini

```
python

import google.generativeai as genai

genai.configure(api_key="tu-api-key")
model = genai.GenerativeModel('gemini-pro')
```

Modelos Open Source (Auto-hospedados)

Llama 3.1 (Meta)

```
python

# Usando Ollama (local)
from langchain.llms import Ollama

llm = Ollama(model="llama3.1:70b")
```

Mistral

```
python

from langchain.llms import HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="mistralai/Mistral-7B-Instruct-v0.2",
    task="text-generation"
)
```

Instalación Ollama (para modelos locales):

```
bash

# Instalar Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Descargar modelo
ollama pull llama3.1:70b
```

3. HERRAMIENTAS PARA APIS Y WEB SCRAPING

Requests (HTTP básico)

```
python

import requests

# Consultar API Catastro
response = requests.get(
    'http://ovc.catastro.meh.es/ovcservweb/OVCSWLocalizacionRC/OVCCallejero.asmx/Con
    params={'Provincia': '28', 'Municipio': '079', 'RC': '1234567VK1234N'}
)
data = response.json()
```

aiohttp (HTTP asíncrono - más rápido)

```
python

import aiohttp
import asyncio

async def consultar_apis_paralelo():
    async with aiohttp.ClientSession() as session:
        # Consultas simultáneas
        task1 = session.get('https://api.catastro.es/...')
        task2 = session.get('https://api.idealista.com/...')
        task3 = session.get('https://api.fotocasa.es/...')

        results = await asyncio.gather(task1, task2, task3)
        return results
```

Beautiful Soup + Scrapy (Web scraping)

```
python

from bs4 import BeautifulSoup
import requests

# Scraping portales inmobiliarios
url = 'https://www.idealista.com/venta-viviendas/madrid/'
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

# Extraer precios
precios = soup.find_all('span', class_='item-price')
```

Para *scraping* avanzado:

```
bash

pip install scrapy selenium playwright
```

Playwright (Scraping JavaScript-heavy sites)

```
python

from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto('https://www.idealista.com/...')

    # Esperar carga dinámica
    page.wait_for_selector('.item-price')
    precios = page.query_selector_all('.item-price')
```

4. BASES DE DATOS

PostgreSQL + pgvector (Vector DB para embeddings)

```
python

import psycopg2
from langchain.vectorstores import PGVector

# Almacenar embeddings de comparables
vectorstore = PGVector(
    connection_string="postgresql://user:pass@localhost:5432/valoraciones",
    embedding_function=embeddings,
    collection_name="comparables"
)

# Buscar similares
comparables = vectorstore.similarity_search(
    "vivienda 95m2 madrid centro",
    k=10
)
```

Pinecone / Weaviate (Vector DB cloud)

```
python

import pinecone

pinecone.init(api_key="tu-key")
index = pinecone.Index("comparables-inmuebles")

# Buscar vectorialmente
results = index.query(
    vector=embedding_inmueble,
    top_k=15,
    include_metadata=True
)
```

MongoDB (Datos no estructurados)

```
python

from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['valoraciones']

# Guardar valoración completa
db.valoraciones.insert_one({
    'referencia_catastral': '1234567VK',
    'valor_tasacion': 265000,
    'comparables': [...],
    'fecha': datetime.now()
})
```

5. PROCESAMIENTO DE DOCUMENTOS

python-docx (Generar informes Word)

```
python

from docx import Document
from docx.shared import Inches

doc = Document()
doc.add_heading('Informe de Valoración IVS', 0)

# Añadir tabla
table = doc.add_table(rows=3, cols=2)
table.cell(0, 0).text = 'Referencia Catastral'
table.cell(0, 1).text = '1234567VK1234N0001AB'

# Añadir imagen
doc.add_picture('mapa_ubicacion.png', width=Inches(5))

doc.save('informe_valoracion.docx')
```

ReportLab (Generar PDFs)

```
python

from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas

pdf = canvas.Canvas("informe.pdf", pagesize=A4)
pdf.setTitle("Informe de Valoración")
pdf.drawString(100, 750, "INFORME DE VALORACIÓN")
pdf.drawString(100, 700, f"Valor: 265.000 €")
pdf.save()
```

PyPDF2 / pdfplumber (Leer PDFs)

```
python

import pdfplumber

# Extraer texto de normativa IVS en PDF
with pdfplumber.open('IVS_2022.pdf') as pdf:
    texto = ''
    for page in pdf.pages:
        texto += page.extract_text()
```

Unstructured ★ (Procesar cualquier documento)

```
python

from unstructured.partition.auto import partition

# Procesa Word, PDF, HTML, etc.
elements = partition(filename="normativa_ivs.pdf")
texto_limpio = "\n\n".join([str(el) for el in elements])
```

6. ANÁLISIS DE DATOS Y CÁLCULO

Pandas (Análisis comparables)

```
python

import pandas as pd

# Cargar comparables
comparables = pd.DataFrame({
    'direccion': [...],
    'superficie': [95, 92, 98, 100],
    'precio': [270000, 265000, 285000, 290000],
    'año': [1985, 1990, 1982, 1988]
})

# Calcular precio/m²
comparables['precio_m2'] = comparables['precio'] / comparables['superficie']

# Estadísticas
media = comparables['precio_m2'].mean()
mediana = comparables['precio_m2'].median()
std = comparables['precio_m2'].std()
```

NumPy (Cálculos matemáticos)

```
python

import numpy as np

# Ajustes por características
ajustes = np.array([1.0, 0.95, 1.02, 0.98]) # antigüedad, reforma, etc.
precio_base = 270000
precio_ajustado = precio_base * np.prod(ajustes)
```

SciPy (Estadística avanzada)

```
python

from scipy import stats

# Test de normalidad de precios
statistic, p_value = stats.shapiro(comparables['precio'])

# Intervalo de confianza
interval = stats.t.interval(0.95, len(comparables)-1,
                             loc=media,
                             scale=stats.sem(comparables['precio']))
```

scikit-learn (Machine Learning para comparables)

```
python

from sklearn.neighbors import NearestNeighbors

# Encontrar comparables más similares
features = comparables[['superficie', 'año', 'latitud', 'longitud']]
model = NearestNeighbors(n_neighbors=10)
model.fit(features)

# Buscar similares al inmueble objetivo
distancias, indices = model.kneighbors([[95, 1985, 40.416, -3.703]])
comparables_cercanos = comparables.iloc[indices[0]]
```

7. VISUALIZACIÓN Y MAPAS**Matplotlib / Seaborn (Gráficos)**

```
python

import matplotlib.pyplot as plt
import seaborn as sns

# Gráfico evolución precios
plt.figure(figsize=(10, 6))
sns.lineplot(data=historico_precios, x='fecha', y='precio_m2')
plt.title('Evolución Precio €/m² - Madrid Centro')
plt.savefig('grafico_evolucion.png')
```

Folium (Mapas interactivos)

```
python

import folium

# Crear mapa con comparables
mapa = folium.Map(location=[40.416, -3.703], zoom_start=15)

# Marcar inmueble objetivo
folium.Marker(
    [40.416, -3.703],
    popup='Inmueble a valorar',
    icon=folium.Icon(color='red')
).add_to(mapa)

# Marcar comparables
for idx, row in comparables.iterrows():
    folium.Marker(
        [row['lat'], row['lon']],
        popup=f"{row['precio']} €",
        icon=folium.Icon(color='blue')
    ).add_to(mapa)

mapa.save('mapa_comparables.html')
```

Plotly (Gráficos interactivos)

```
python

import plotly.express as px

fig = px.scatter_mapbox(
    comparables,
    lat='latitud',
    lon='longitud',
    size='superficie',
    color='precio_m2',
    hover_name='direccion',
    mapbox_style='open-street-map',
    zoom=14
)
fig.write_html('mapa_interactivo.html')
```

8. APIS ESPECÍFICAS DEL SECTOR INMOBILIARIO

Catastro España

```
python

import requests

def consultar_catastro(referencia):
    url = 'http://ovc.catastro.meh.es/ovcswweb/OVCSWLocalizacionRC/OVCCallejero.as
    params = {
        'Provincia': referencia[:2],
        'Municipio': referencia[2:5],
        'RC': referencia
    }
    response = requests.get(url, params=params)
    return response.text # XML
```

Idealista API (Requiere partnership)

```
python

import requests
from requests_oauthlib import OAuth2Session

# Autenticación OAuth2
oauth = OAuth2Session(client_id, token=token)
response = oauth.get('https://api.idealista.com/3.5/es/search')
```

Google Maps / Geocoding

```
python

import googlemaps

gmaps = googlemaps.Client(key='tu-api-key')

# Geocodificar dirección
geocode = gmaps.geocode('Calle Mayor 45, Madrid')
lat = geocode[0]['geometry']['location']['lat']
lon = geocode[0]['geometry']['location']['lng']

# Lugares cercanos
places = gmaps.places_nearby(
    location=(lat, lon),
    radius=500,
    type='subway_station'
)
```


OpenStreetMap (Gratuito)

```
python

from geopy.geocoders import Nominatim

geolocator = Nominatim(user_agent="valoracion_app")
location = geolocator.geocode("Calle Mayor 45, Madrid")
print(f"Coordenadas: {location.latitude}, {location.longitude}")
```

9. ORQUESTACIÓN Y DEPLOYMENT

FastAPI ★ (Backend del agente)

```
python

from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel

app = FastAPI()

class SolicitudValoracion(BaseModel):
    referencia_catastral: str
    objeto: str
    email_cliente: str

@app.post("/valoracion/solicitar")
async def crear_valoracion(solicitud: SolicitudValoracion, background_tasks: BackgroundTasks):
    # Lanzar agente en background
    background_tasks.add_task(ejecutar_agente_valoracion, solicitud)
    return {"mensaje": "Valoración iniciada", "id": "2024-0156"}

async def ejecutar_agente_valoracion(solicitud):
    # Aquí va la orquestación del agente
    resultado = agente_coordinador.run(solicitud)
    enviar_email(resultado)
```

Instalación:

```
bash

pip install fastapi uvicorn
uvicorn main:app --reload
```

Celery (Tareas asíncronas)

```
python

from celery import Celery

app = Celery('valoraciones', broker='redis://localhost:6379')

@app.task
def procesar_valoracion(referencia):
    # Tarea pesada en background
    datos_catastro = obtener_catastro(referencia)
    comparables = buscar_comparables(datos_catastro)
    valor = calcular_valoracion(comparables)
    return valor
```

Docker (Containerización)

```
dockerfile

FROM python:3.11

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

LangServe (Deploy agentes LangChain)

```
python

from langchain.agents import AgentExecutor
from langserve import add_routes

# Añadir agente como API
add_routes(
    app,
    AgentExecutor(...),
    path="/agente-valoracion"
)
```

9. ORQUESTACIÓN Y DEPLOYMENT

FastAPI ★ (Backend del agente)

```
python

from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel

app = FastAPI()

class SolicitudValoracion(BaseModel):
    referencia_catastral: str
    objeto: str
    email_cliente: str

@app.post("/valoracion/solicitar")
async def crear_valoracion(solicitud: SolicitudValoracion, background_tasks: BackgroundTasks):
    # Lanzar agente en background
    background_tasks.add_task(ejecutar_agente_valoracion, solicitud)
    return {"mensaje": "Valoración iniciada", "id": "2024-0156"}

async def ejecutar_agente_valoracion(solicitud):
    # Aquí va la orquestación del agente
    resultado = agente_coordinador.run(solicitud)
    enviar_email(resultado)
```

Instalación:

```
bash

pip install fastapi uvicorn
uvicorn main:app --reload
```

Celery (Tareas asíncronas)

```
python

from celery import Celery

app = Celery('valoraciones', broker='redis://localhost:6379')

@app.task
def procesar_valoracion(referencia):
    # Tarea pesada en background
    datos_catastro = obtener_catastro(referencia)
    comparables = buscar_comparables(datos_catastro)
    valor = calcular_valoracion(comparables)
    return valor
```

Docker (Containerización)

```

dockerfile

FROM python:3.11

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

LangServe (Deploy agentes LangChain)

```

python

from langchain.agents import AgentExecutor
from langserve import add_routes

# Añadir agente como API
add_routes(
    app,
    AgentExecutor(...),
    path="/agente-valoracion"
)

```

10. MONITOREO Y LOGGING**LangSmith ★ (Monitoring para LangChain)**

```

python

import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "tu-api-key"

# Automáticamente trackea todas las llamadas

```

Weights & Biases

```

python

import wandb

wandb.init(project="valoracion-inmuebles")

# Log métricas
wandb.log({
    "tiempo_proceso": 25.3,
    "num_comparables": 15,
    "valor_final": 265000
})

```

Loguru (Logging mejorado)

```
python

from loguru import logger

logger.add("valoraciones.log", rotation="1 week")

logger.info(f"Iniciando valoración {referencia}")
logger.success(f"Valoración completada: {valor} €")
logger.error(f"Error en API Catastro: {error}")
```

11. TESTING Y VALIDACIÓN

Pytest

```
python

import pytest

def test_calculo_valoracion():
    comparables = [270000, 265000, 285000]
    resultado = calcular_valor_medio(comparables)
    assert resultado == 273333.33

def test_validacion_ivs():
    informe = generar_informe(datos)
    assert validar_cumplimiento_ivs(informe) == True
```

Guardrails AI (Validación de outputs LLM)

```
python

from guardrails import Guard
import guardrails as gd

guard = Guard.from_string(
    validators=[
        gd.validators.ValidRange(min=50000, max=5000000),
        gd.validators.ValidFormat(format="number")
    ]
)

# Valida que el LLM devuelva un precio razonable
validated_output = guard.parse(llm_output)
...

---
```

```

## STACK TECNOLÓGICO RECOMENDADO

### **Stack Completo para Producción:**
...

Frontend (opcional):
├─ React/Next.js (interfaz web)
└─ Streamlit (prototipado rápido)

Backend:
├─ FastAPI (API REST)
├─ LangChain + CrewAI (Agentes)
├─ Claude Sonnet (LLM principal)
└─ Celery + Redis (tareas async)

Datos:
├─ PostgreSQL + pgvector (BD principal)
├─ MongoDB (logs, documentos)
└─ Pinecone (búsqueda vectorial)

Herramientas:
├─ python-docx + ReportLab (informes)
├─ Playwright (scrapping)
├─ Pandas + NumPy (análisis)
└─ Folium/Plotly (visualización)

APIs Externas:
├─ Catastro España
├─ Google Maps
├─ Idealista/Fotocasa
└─ DocuSign (firma)

DevOps:
├─ Docker + Docker Compose
├─ GitHub Actions (CI/CD)
└─ AWS/Google Cloud (hosting)

Monitoring:
├─ LangSmith (trazabilidad agentes)
├─ Sentry (errores)
└─ Loguru (logs)

```

INSTALACIÓN RÁPIDA (Comando Único)

```

bash

# Crear entorno virtual
python -m venv venv
source venv/bin/activate # Linux/Mac
# venv\Scripts\activate # Windows

```



```
# Instalar todo
pip install \
  langchain langchain-openai langchain-anthropic \
  crewai crewai-tools \
  fastapi uvicorn \
  pandas numpy scipy scikit-learn \
  python-docx reportlab pdfplumber \
  requests aiohttp beautifulsoup4 scrapy playwright \
  psycpg2-binary pymongo pinecone-client \
  folium plotly matplotlib seaborn \
  googlemaps geopy \
  celery redis \
  pytest guardrails-ai \
  loguru python-dotenv

# Instalar Playwright browsers
playwright install
```

RECURSOS DE APRENDIZAJE

Documentación oficial:

- LangChain: <https://python.langchain.com/>
- CrewAI: <https://docs.crewai.com/>
- Anthropic Claude: <https://docs.anthropic.com/>

Cursos:

- DeepLearning.AI: "LangChain for LLM Application Development"
- CrewAI Crash Course (YouTube)

Comunidades:

- Discord de LangChain
- Reddit: r/LangChain, r/LocalLLaMA